Sunum Anlatım Rehberi: Slayt Slayt Anlatım ve Olası Hoca Soruları

Her slayt için üç şey var: **(1) Önce sen anla**, **(2) Sunumda ne söyle**, **(3) Hoca ne sorabilir ve kısa cevabı**. Kaynak numaraları sunumdakiyle aynı: [1] Subramanian et al. 2021, [2] Fridrich et al. 2007, [3] Bao et al. 2018, [4] Fan et al. 2011.

Anlatının ana hikâyesi (hepsini bu çizgiye bağla): Bir yöntem geliyor → bir zayıflığı yüzünden yakalanıyor → sonraki yöntem o zayıflığı düzeltiyor. LSB → JSteg → Outguess → F3 → F4 → F5 → matrix embedding → wet paper codes/STC → kayıplı kanalda dayanıklılık.

Dürüstlük notu: Slaytlardaki sayısal örneklerin bir kısmı makalelerde birebir var, bir kısmı makaledeki **kuralın** benim hazırladığım sayısal örneği. Hocaya "kural makalede, sayıları örnek olsun diye ben seçtim" demen yeterli. Hangisi hangisi:

- **Makalede birebir:** Fan et al. (1,7,3,2) örneği ve hash hesapları [4]; genişletme katmanları tablosu [4]; p ve α_p eşlemesi (0.2→4, 0.15→5, 0.1→5, 0.09→6, 0.05→7) [2]; 3×7 H matrisi [2]; STC hata yayılımı formülleri [3]; Bao'nun yazma-geri kuralı ve eşikleri [3]; verimlilik tablosu [4]; hata olasılığı tablosu [2].
- **Kural makalede, sayılar benim:** LSB pikselleri (150, 201, 78); F3 ve F4 örnekleri; matrix embedding adım adım örneği (x = 1011001, m = 011); (1,3,2) hücresi örneği (a = 101, w = 11).

Düzeltme notu: Çalışma özetinde ve JSteg slaytında "değer çiftlerinin sayıları birbirine yaklaşır" açıklamasını anlattım. Makalelerde sadece "POV (pairs of values) problemi χ^2 testiyle yakalanır" ifadesi geçiyor [4]; "çiftlerin eşitlenmesi" mekanizması standart steganaliz bilgisi, bu makalelerde yazmıyor. Hoca sorarsa "bu açıklama genel steganaliz bilgisi, makaledeki ifade POV problemi ve χ^2 testi" de.

Slayt 1: Kapak

Önce sen anla. Bu sunum bir literatür taraması: LSB'den F5'e kadar fikirler nasıl gelişti ve nerede tıkandı.

Sunumda söyle. "Bu sunumda görüntü steganografisinde LSB'den başlayıp JPEG tabanlı F5'e ve matrix embedding'e kadar gelen yöntemleri, her birinin nasıl çalıştığını ve neden yetersiz kaldığını anlatacağım. Her yöntemi küçük bir örnekle göstereceğim."

Hoca sorabilir:

- *Bu çalışma tezinle nasıl bağlantılı?* Tez konun steganografi ve cebirsel kodlama teorisi; matrix embedding tam bu iki alanın kesişimi (Hamming kodları ile gömme). Bu yüzden F5'ten sonra kodlama kısmına yavaşlıyorum.

Slayt 2: Hiding the message itself

Önce sen anla. Kriptografi içeriği saklar, steganografi **varlığı**. Steganografi "kırıldı" sayılması için mesajın okunması gerekmez, sadece mesajın var olduğunun anlaşılması yeter [2], [4]. Formel olarak: gömme `y = Emb(x, m, k)`, çıkarma `Ext(y, k) = m` [2].

Sunumda söyle. "Kriptografide şifreli metin ortada görünür ve şüphe çeker [1]. Steganografide amaç iletişimin olduğunu bile gizlemek. Gömme fonksiyonu cover, mesaj ve anahtardan stego üretir, çıkarma fonksiyonu aynı anahtarla mesajı geri verir."

Hoca sorabilir:

- *İkisi birlikte kullanılabilir mi?* Evet; [1]'de bir çalışma (Patel & Meena) LSB gömmesini kriptografi ile birleştiriyor, ayrıca konum seçimi anahtara bağlı yapılıyor.
- *Neden "varlığı tespit edilirse kırıldı" deniyor?* Çünkü hedef şüphe çekmeden iletişim; şüphe oluştuysa gözlemci mesajı okuyamasa bile amaç kaybolmuş.
- *Watermarking'den farkı?* Steganografi gizliliğe, watermarking dayanıklılığa odaklanır [1].

Slayt 3: Embedding pipeline

Önce sen anla. Her steganografik sistem aynı iskelete sahip: cover seç → mesajı bite çevir → alan seç → konum seç → gömme işlemi → gönder; alıcı aynı anahtarla çıkarır. Yöntemleri birbirinden ayıran şey 3., 4. ve 5. adımlar [2], [3], [4].

Sunumda söyle. "Birazdan göreceğimiz her yöntem bu hattın farklı adımlarını değiştiriyor: LSB sadece piksel alanında basit bir değiştirme yapıyor, JSteg alanı DCT'ye taşıyor, F5 gömme işlemini ve kodlamayı değiştiriyor, STC konum seçimini maliyete bağlıyor."

Hoca sorabilir:

- *Kapasite ve verimlilik farkı nedir?* Kapasite toplam gömülebilen bit; verimlilik değişiklik başına gömülen bit [2], [4].
- *Hangi ölçütlerle değerlendirilir?* Kapasite (bpp/bpac), PSNR/MSE, tespit hatası P [1], [2].

Slayt 4: LSB: simple, and easy to catch

Önce sen anla. LSB: piksellerin son bitini mesaj bitiyle değiştir. Rastgele verilerde bir pozisyonun değişme olasılığı %50, bu yüzden değişiklik başına 2 bit gömülür; resmi doldurursan verinin yarısı değişir ve istatistiksel olarak yakalanır [4]. Zayıflıklar: aşırı yüklemede görünür iz [1], piksel alanı saldırılara açık [4], LSB çevirmenin kendisi tespit edilebilir [2], JPEG sıkıştırma mesajı bozar [3].

Sunumda söyle. "LSB basit ve kapasitesi yüksek ama güvenliği zayıf. Ayrıca kayıplı bir kanaldan geçerse mesaj kaybolur; bu yüzden sosyal medya gibi ortamlarda işe yaramıyor."

Hoca sorabilir:

- *Neden 2 bit/değişiklik?* Mesaj ve cover bitleri düzgün dağılıyorsa gömülecek bit yarı olasılıkla zaten aynı; ortalama yarı pozisyon değişir [4].
- *Varyantlar neler?* k-LSB, Huffman+LSB, RGB 2:2:4, LSB+PVD, anahtarlı rastgele konum (dinamik anahtar) [1].
- *Dinamik LSB tam olarak ne?* Makalelerde tek bir standart algoritma olarak tanımlı değil; [1]'de Patel & Meena: PRNG ile piksel seçimi ve anahtar şifreleme. Ben "anahtara bağlı rastgele konum" fikri olarak anlattım.

Slayt 5: LSB by example

Önce sen anla. Üç piksel: 150 (10010110), 201 (11001001), 78 (01001110). Mesaj 1, 0, 1. Sadece son bit değişiyor: 151, 200, 79. Alıcı son bitleri okuyunca 1, 0, 1'i alır. Her değer en fazla 1 değişir.

Sunumda söyle. "Burada üç gri piksele üç bit gömüyorum. Her piksel en fazla 1 birim değişiyor, göz bunu fark etmiyor. Ama değişikliklerin istatistiksel kalıbı steganaliz için ipucu veriyor." (Not: sayılar örnek, kural [1]'den.)

Hoca sorabilir:

- *Piksel zaten istenen bitteyse?* Değişiklik yapılmaz.
- *Rastgele mi sıralı mı gömülür?* İkisi de olur; anahtarlı rastgele yol daha güvenli bir seçenek [1], [2].

Slayt 6: Spatial vs transform domain

Önce sen anla. Spatial: piksel LSB'leri. Transform: JPEG'in 8×8 blok DCT katsayıları. JPEG hattı: renk dönüşümü, örnekleme, 8×8 bloklar, DCT, kuantizasyon, kodlama [3]. Gömme **kuantize edilmiş** katsayılara yapılır. F5 sadece sıfır olmayan AC katsayılarına gömer [2].

Sunumda söyle. "Piksel alanında yapılan gömme piksel değişimine ve görsel saldırıya açık [4]. JPEG en yaygın format olduğu için çalışmalar DCT katsayılarına gömmeye yoğunlaştı [2]."

Hoca sorabilir:

- *DC ve AC nedir?* DC blok ortalaması, 63 AC katsayı ayrıntıyı taşır; makalede AC modlarından bahsediliyor (MBS 63 mod için model kuruyor) [2].

- *Kuantizasyon neden önemli?* Katsayılar tam sayıya yuvarlanır; mesaj bu tam sayıların LSB'sine yazılır, dosya olarak saklanan değer budur [3].

Slayt 7: From JSteg to F5 (özet tablo)

Önce sen anla. Bu slayt haritadır: beş yöntemin bir cümlelik fikri ve tek cümlelik sorunu. Detayları sonraki slaytlarda.

Sunumda söyle. "Önce haritayı göstereyim, sonra her yöntemi tek tek ve örnekle açıyorum."

Hoca sorabilir:

- *Neden bu sırayla?* Kronolojik ve her biri öncekinin zayıflığını düzeltiyor [4].

Slayt 8: JSteg

Önce sen anla. JSteg, LSB'nin JPEG sürümü. 0 ve 1 değerli katsayılar atlanır çünkü bu ikisi bir LSB çiftidir ve 0'ı değiştirmek büyük bozulma yaratır [2]. Yol önce sıralı, sonra sözde rastgele. Sorun: DCT histogramında karakteristik izler [2]; değer çiftleri (POV) sorunu χ^2 testiyle yakalanır [4]; çok düşük oranlarda bile kategori saldırısıyla bulunur [2].

Sunumda söyle. "JSteg tam bir LSB kopyası, sadece alan DCT katsayıları. Histogramın birinci derece istatistiği bozuluyor ve basit testlerle yakalanıyor."

Hoca sorabilir:

- *χ^2 testi nasıl çalışır?* Makalelerde detay yok; sadece POV sorununu yakaladığı yazıyor [4]. Fikir: iki ardışık değerin sayılarının aşırı yakın çıkması şüpheli. (Bu açıklama genel bilgi.)
- *Neden 0 ve 1 atlanıyor?* Aralarında LSB çifti olduğu ve 0'a dokunmak büyük bozulma getirdiği için [2].

Slayt 9: Outguess

Önce sen anla. v0.1: gömme konumlarını PRNG ile tüm DCT alanına dağıtıyor. χ^2 testi başta işe yaramadı ama sonra istatistiksel izler bulundu [4]. v0.2: gömmeden sonra istatistikleri düzeltmeye çalışıyor (statistical restoration) [4]. Fikir: cover'ı ikiye böl, birinde göm, diğerinde histogramı düzelt [2]. Sorun: 8×8 blok sınırlarındaki süreksizlikle gömülen verinin uzunluğu tahmin edilebiliyor [4]; sadece histogramı onarmak yetmez, blok içi ve bloklar arası bağımlılıklar da var; hatta tespit edilebilirliği artırabilir [2].

Sunumda söyle. "Outguess iki fikir getirdi: rastgele dağıtmak ve istatistiği onarmak. Ama onarım sadece birinci derece histogramı kapsıyor, blok yapısını bozmaya devam ediyor."

Hoca sorabilir:

- *Statistical restoration neden işe yaramadı?* Çünkü DCT katsayılarının karmaşık bağımlılıkları var; sadece histogramı onarmak yeterli değil [2].

Slayt 10: F3

Önce sen anla. F3 LSB'yi çevirmiyor, **mutlak değeri 1 azaltıyor**. Mesaj biti katsayı ile aynıysa değişiklik yok, değilse |katsayı| − 1 [4]. Bu JSteg'in POV sorununu çözüyor. Sorun: çift katsayılar çoğalıyor, histogram doğal görünmüyor [4]. Slayttaki tablo (3, −2, 5, 1) kuralın örneği; 1'in 0 olması shrinkage'ın habercisi.

Sunumda söyle. "F3'te amaç LSB çevirmenin bıraktığı izden kurtulmak. Ama azaltma işlemi katsayı değerlerini sıfıra doğru kaydırıyor ve histogramda çift değerlerin fazlalığı oluşuyor."

Hoca sorabilir:

- *Neden "azaltma" çevirmeden daha iyi?* [2], yüksek tespit edilebilirliğin sebebini LSB çevirmenin kendisine bağlıyor; azaltma bu simetrik çevirmeyi yapmıyor.
- *Katsayı 0 olursa ne olur?* Alıcı sıfırları okumaz; bu shrinkage (F5 slaytı) [2].

Slayt 11: F4

Önce sen anla. F4 bit değerini katsayının **işaretine** bağlıyor: çift-negatif ve tek-pozitif katsayı 1, tek-negatif ve çift-pozitif 0. Bit sembole eşitse değişiklik yok, değilse |katsayı| − 1 [4]. Sonuç: histogram temiz bir görüntüye istatistiksel olarak benziyor [4].

Sunumda söyle. "F4, F3'ün histogram sorununu işaret tabanlı eşleme ile çözüyor. Örnek: 4 sayısı 0 temsil eder, 1 gömmek için 3 yapılır; 3 tek-pozitif, yani 1."

Hoca sorabilir:

- *F4'ün zayıflığı ne?* Makalelerde F4 için açık bir zayıflık yazmıyor; F5'in matrix embedding getirmesi gömme verimliliği hedefinin devamı olarak okunabilir (bu yorum benim).
- *Negatif sayılar nasıl işleniyor?* İşaret eşlemesiyle: tek-negatif 0, çift-negatif 1 [4].

Slayt 12: F5

Önce sen anla. Üç parça: (1) |katsayı| azaltma, sadece sıfır olmayan AC katsayıları [2]; (2) matrix embedding (Hamming) [2], [4]; (3) permutative straddling: mesajı tüm cover'a dağıtır [4]. **Shrinkage:** ±1 katsayı 0 olursa alıcı sıfırı atlar, bu yüzden aynı bit sonraki

katsayıya yeniden gömülür; verimlilik düşer, histogram etkilenir. F5, ±1'lerin yaklaşık %50'sini atlayarak etkiyi azaltır [2]. −F5 (azaltmak yerine artırmak) beklenmedik şekilde F5'ten kötü; nsF5 (wet paper codes) en iyisi [2].

Sunumda söyle. "F5'in asıl yeniliği matrix embedding: aynı mesajı daha az değişiklikle gömmek. Shrinkage bu yöntemin yan etkisi; nsF5 bunu wet paper codes ile çözüyor."

Hoca sorabilir:

- *Shrinkage neden sorun?* Alıcı sıfırları atlıyor, bu yüzden gönderici aynı biti tekrar gömmek zorunda; fazladan değişiklik demek [2].
- *−F5 neden kötü?* Makale bunu F5 tasarımcısının seçimini doğrulayan sürpriz bir sonuç olarak veriyor; sebep derinlemesine açıklanmıyor [2].
- *Permutative straddling nedir?* Mesajı tüm taşıyıcıya dağıtarak güvenliği artıran teknik [4].

Slayt 13: F5's key idea: matrix embedding

Önce sen anla. 7 koeffisiyent LSB'sine 3 bit, en fazla 1 değişiklikle. H matrisinin sütunları 1'den 7'ye kadar ikili sayılar [2]. Verimlilik E = (n+1)/n · k; (7,3) için 3.43 bit/değişiklik [4]. Bedeli: oran düşük (k/n azalıyor) [4]. H, Hamming kodunun parite-kontrol matrisi (bu etiket makale dışı genel bilgi; [2] "binary Hamming codes" diyor).

Sunumda söyle. "Matrix embedding sendrom kodlamasıdır: mesaj, cover bitlerinin sendromu olur. Cover'ı değiştirmeden sendrom mesaja eşitse hiç değişiklik gerekmiyor; değilse tek bit çevirmek yetiyor."

Hoca sorabilir:

- *Hamming (7,4) ile ilişkisi?* Burada kullanılan H, Hamming(7,4) kodunun parite-kontrol matrisi; Hamming kodu tek hata düzeltir, burada "hata" yerine bilinçli tek bit değişimi var. (Genel kodlama teorisi bilgisi; bir ders kitabına atıf ekle.)
- *Verimlilik neden artıyor?* Daha büyük k ile aynı tek değişiklik daha fazla bit taşıyor, ama hücre uzunluğu 2^k − 1 olarak üstel büyüyor [4].